AI Raw Response

```
content = {
  "public/index.html": """<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Mi Sitio Estático</title>
  <meta name="description" content="Sitio estático tipo WP con JSON" />
  <link rel="stylesheet" href="/public/assets/style.css" />
  <link rel="icon" href="/public/assets/logo.svg" />
  <link rel="sitemap" type="application/xml" title="Sitemap" href="/sitemap.xml" />
  <meta name="robots" content="index, follow" />
</head>
<body>
  <header class="site-header" id="site-header">
    <div class="brand">
      <img src="/public/assets/logo.svg" alt="logo" id="brand-logo" />
      <h1 id="brand-title">Mi Sitio Estático</h1>
    </div>
    <nav>
      <a href="/" data-link>Inicio</a>
      <a href="/blog" data-link>Blog</a>
      <a href="/contacto" data-link>Contacto</a>
      <a href="?admin=1" id="admin-link">Admin</a>
    </nav>
  </header>

  <main id="app" tabindex="0">
    <section class="loading">Cargando...</section>
  </main>

  <footer id="site-footer">
    <div class="footer-inner">
      <p id="footer-text">© Mi Sitio Estático</p>
      <small id="footer-note">Generado con JSON</small>
    </div>
  </footer>

  <!-- Third-party helpers (CDN) -->
  <script src="https://cdn.jsdelivr.net/npm/ajv@8/dist/ajv7.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/dompurify@2.4.3/dist/purify.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>

  <!-- Application scripts -->
  <script type="module" src="/src/main.js"></script>
</body>
</html>
""",
  "public/assets/style.css": """:root{
  --bg:#ffffff; --fg:#0b1220; --muted:#6b7280; --accent:#1f6feb; --success:#059669;
--danger:#dc2626;
}
*{box-sizing:border-box}
body{font-family:system-ui,-apple-system,Segoe UI,Roboto,"Helvetica Neue",Arial; margin:0;
color:var(--fg); background:var(--bg); line-height:1.5}
a{color:var(--accent); text-decoration:none}
a:focus{outline:3px solid #bfdbfe; outline-offset:2px}
.site-header{display:flex;justify-content:space-between;align-items:center;padding:12px
20px;border-bottom:1px solid #eee;background:#f8fafc}
.brand{display:flex;align-items:center;gap:12px}
```

```
.brand img{width:40px;height:40px}
.brand h1{font-size:1.1rem;margin:0}
nav a{margin-left:12px;font-weight:600}
main{padding:24px;max-width:1100px;margin:0 auto}
.hero{display:flex;gap:20px;align-items:center;padding:36px;border:1px solid
#eef2ff;background:linear-gradient(180deg, #ffffff, #f8fdff);border-radius:8px}
.hero .left{flex:1}
.hero .cta{background:var(--accent);color:white;padding:10px
14px;border-radius:6px;border:none;cursor:pointer}

.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:16px;margi
.card{border:1px solid #e6edf3;padding:12px;border-radius:8px;background:white}
.post-list{margin-top:18px}
.post-item{padding:12px;border-bottom:1px dashed #eef3f8}
.controls{display:flex;gap:8px;flex-wrap:wrap;margin-top:12px}
.btn{background:#111827;color:white;padding:8px 12px;border-radius:6px;border:none;cursor:
.btn.ghost{background:transparent;color:var(--accent);border:1px solid #e2e8f0}
.form-row{display:flex;flex-direction:column;margin-bottom:10px}
label{font-size:0.9rem;margin-bottom:6px;color:var(--muted)}
input[type="text"],input[type="email"],textarea,select{padding:8px;border:1px solid
#cbd5e1;border-radius:6px;font-size:1rem}
textarea{min-height:120px;resize:vertical}
.admin-badge{background:#fff7ed;color:#92400e;padding:6px 10px;border-radius:999px;border:
#ffedd5;margin-left:12px}

.footer-inner{display:flex;justify-content:space-between;align-items:center;padding:18px;b
solid #eee;background:#fafafa}
.kv{font-family:monospace;background:#0f172a;color:#fff;padding:4px 6px;border-radius:4px}
.loading{padding:40px;text-align:center;color:var(--muted)}

.sr-only{position:absolute;width:1px;height:1px;padding:0;margin:-1px;overflow:hidden;clip
.preview{border:1px dashed #cbd5e1;padding:12px;border-radius:6px;background:#fff}
.small{font-size:0.9rem;color:var(--muted)}
.error{color:var(--danger)}
.success{color:var(--success)}
""",
  "public/assets/logo.svg": """<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 100 10
width="100" height="100" rx="18" fill="#1f6feb"/><text x="50" y="55" font-size="44"
text-anchor="middle" font-family="Verdana" fill="#fff">M</text></svg>""",
  "public/robots.txt": """User-agent: *
Allow: /
Sitemap: /sitemap.xml
""",
  "public/sitemap.xml": """<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url><loc>/</loc></url>
  <url><loc>/blog</loc></url>
  <url><loc>/contacto</loc></url>
</urlset>
""",
  "src/main.js": """import { loadConfig, loadRoutes, isAdminMode } from './lib/config.js';
import { renderHome } from './pages/home.js';
import { renderBlogList, renderBlogDetail } from './pages/blog.js';
import { renderContact } from './pages/contact.js';
import { renderAdmin } from './admin/index.js';

const app = document.getElementById('app');
const headerTitle = document.getElementById('brand-title');
const logoImg = document.getElementById('brand-logo');
const footerText = document.getElementById('footer-text');
```

```
let CONFIG = null;
let ROUTES = null;

async function init(){
  try{
    CONFIG = await loadConfig();
    ROUTES = await loadRoutes();
    setupBranding(CONFIG);
    handleRouting();
    window.addEventListener('popstate', handleRouting);
    document.addEventListener('click', onLinkClick);
  }catch(err){
    app.innerHTML = `<div class="error">Error cargando datos: ${err.message}</div>`;
    console.error(err);
  }
}

function setupBranding(cfg){
  if(!cfg) return;
  headerTitle.textContent = cfg.siteName || 'Mi Sitio Estático';
  if(cfg.brand && cfg.brand.logo) logoImg.src = cfg.brand.logo;
  if(cfg.brand && cfg.brand.primaryColor){
    document.documentElement.style.setProperty('--accent', cfg.brand.primaryColor);
  }
  if(cfg.footer){
    footerText.textContent = cfg.footer;
  }else{
    footerText.textContent = cfg.siteName || 'Mi Sitio Estático';
  }
  // show admin link if enabled
  const adminLink = document.getElementById('admin-link');
  if(cfg.admin && cfg.admin.enabled){
    adminLink.style.display = '';
  }else{
    adminLink.style.display = 'none';
  }
}

function onLinkClick(e){
  const a = e.target.closest('a[data-link]');
  if(!a) return;
  e.preventDefault();
  const href = a.getAttribute('href');
  history.pushState({}, '', href);
  handleRouting();
}

function matchRoute(path){
  // simple matching using routes.json order
  const routes = ROUTES.routes || [];
  for(const r of routes){
    if(r.path.includes(':')){
      // param route like /blog/:slug
      const base = r.path.split('/:')[0];
      if(path.startsWith(base) && path.length > base.length+1) return {route:r, params:{sl
path.substring(base.length+1)}};
    }else if(r.path === path){
      return {route:r, params:{}};
    }
  }
  return null;
```

```
}

async function handleRouting(){
  const url = new URL(window.location.href);
  const path = url.pathname === '' ? '/' : url.pathname;
  // admin mode via query param and password
  const adminMode = isAdminMode(url.searchParams, CONFIG);

  // if admin param is present, open admin UI
  if(adminMode && url.searchParams.get('admin') === '1'){
    app.innerHTML = '';
    await renderAdmin({mount:app, config:CONFIG});
    document.getElementById('site-header').classList.add('admin-badge');
    return;
  }else{
    document.getElementById('site-header').classList.remove('admin-badge');
  }

  const match = matchRoute(path);
  if(!match){
    app.innerHTML = '<h2>404 - No encontrado</h2><p>La ruta no existe.</p>';
    return;
  }
  const page = match.route.page;
  if(page === 'home'){
    app.innerHTML = '';
    await renderHome({mount:app, config:CONFIG});
  }else if(page === 'blog-list'){
    app.innerHTML = '';
    await renderBlogList({mount:app});
  }else if(page === 'blog-detail'){
    app.innerHTML = '';
    await renderBlogDetail({mount:app, slug: match.params.slug});
  }else if(page === 'contact'){
    app.innerHTML = '';
    await renderContact({mount:app, config:CONFIG});
  }else{
    app.innerHTML = '<h2>Página</h2><p>Página sin renderizador.</p>';
  }

  // update SEO title
  if(CONFIG && CONFIG.seo && CONFIG.seo.title){
    document.title = CONFIG.seo.title;
  }
}

init();
""",
  "src/lib/config.js": """export async function loadConfig(){
  const res = await fetch('/data/config.json');
  if(!res.ok) throw new Error('No se pudo cargar /data/config.json');
  return await res.json();
}

export async function loadRoutes(){
  const res = await fetch('/data/routes.json');
  if(!res.ok) throw new Error('No se pudo cargar /data/routes.json');
  return await res.json();
}

export function isAdminMode(searchParams, config){
```

```
  if(!config || !config.admin || !config.admin.enabled) return false;
  const adminFlag = searchParams.get('admin');
  if(!adminFlag) return false;
  const password = searchParams.get('password') || '';
  return password === config.admin.password;
}
""",
  "src/lib/storage.js": """export async function listPostsIndex(){
  const res = await fetch('/data/posts/index.json');
  if(!res.ok) throw new Error('No se pudo cargar posts index');
  return res.json();
}

export async function loadPostByFile(filename){
  const res = await fetch('/data/posts/' + filename);
  if(!res.ok) throw new Error('No se pudo cargar post ' + filename);
  return res.json();
}

export async function loadPostBySlug(slug){
  const idx = await listPostsIndex();
  const entry = idx.posts.find(p => p.slug === slug);
  if(!entry) throw new Error('Post no encontrado: ' + slug);
  return loadPostByFile(entry.file);
}

// Utility for admin in-browser: triggers download to simulate 'save'
export function downloadJSON(filename, obj){
  const blob = new Blob([JSON.stringify(obj, null, 2)], {type:'application/json'});
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  a.click();
  URL.revokeObjectURL(url);
}
""",
  "src/lib/schema.js": """export async function loadSchema(name){
  const res = await fetch('/schemas/' + name + '.json');
  if(!res.ok) throw new Error('No se pudo cargar esquema ' + name);
  return res.json();
}""",
  "src/pages/home.js": """import { marked } from 'marked';

export async function renderHome({mount, config}){
  const el = document.createElement('div');
  el.className = 'home';
  const blocks = (config && config.home && config.home.blocks) || [];
  // hero first
  for(const block of blocks){
    if(block.type === 'hero'){
      const hero = document.createElement('section');
      hero.className = 'hero';
      hero.innerHTML = `
        <div class="left">
          <h2>${escapeHtml(block.title || 'Bienvenido')}</h2>
          <p class="small">${escapeHtml(block.subtitle || '')}</p>
          ${block.cta ? `<button class="cta" onclick="location.href='${block.cta.link ||
'/'}'">${escapeHtml(block.cta.text || 'Saber más')}</button>` : ''}
        </div>
        <div class="right">${block.image ? `<img src="${block.image}"
```

```
alt="${escapeHtml(block.title||'')}" style="max-width:220px;border-radius:8px" />` : ''}</
      `;
      el.appendChild(hero);
    }else if(block.type === 'cards'){
      const grid = document.createElement('div');
      grid.className = 'grid';
      for(const it of (block.items || [])){
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `<h3>${escapeHtml(it.title)}</h3><p class="small">${escapeHtml(it
|| '')}</p><div class="controls"><a data-link href="${it.link}">Ir</a></div>`;
        grid.appendChild(card);
      }
      el.appendChild(grid);
    }else if(block.type === 'posts'){
      // list latest posts preview (blocks can define limit)
      const postsSection = document.createElement('section');
      postsSection.innerHTML = `<h3>${escapeHtml(block.title || 'Últimos posts')}</h3><div
id="home-posts" class="post-list">Cargando posts...</div>`;
      el.appendChild(postsSection);
      // load posts
      import('../lib/storage.js').then(m=>{
        m.listPostsIndex().then(async idx=>{
          const listEl = el.querySelector('#home-posts');
          listEl.innerHTML = '';
          const limit = block.limit || 3;
          const files = idx.posts.slice(0, limit);
          for(const f of files){
            const p = await m.loadPostByFile(f.file);
            const item = document.createElement('div');
            item.className = 'post-item';
            item.innerHTML = `<strong><a href="/blog/${p.slug}"
data-link>${escapeHtml(p.title)}</a></strong><div class="small">${escapeHtml(p.excerpt ||
'')}</div>`;
            listEl.appendChild(item);
          }
        }).catch(err=>{console.error(err);});
      });
    }else{
      // raw HTML/markdown block
      const section = document.createElement('section');
      if(block.content){
        if(block.contentFormat === 'markdown'){
          section.innerHTML = marked.parse(block.content || '');
        }else{
          section.innerHTML = block.content || '';
        }
      }
      el.appendChild(section);
    }
  }

  mount.appendChild(el);
}

function escapeHtml(s){ return String(s||'').replace(/[&<>"']/g, function(m){return
{'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;","'":'&#39;'}[m];}); }""",
  "src/pages/blog.js": """import { listPostsIndex, loadPostByFile, loadPostBySlug } from
'../lib/storage.js';
import DOMPurify from 'https://cdn.jsdelivr.net/npm/dompurify@2.4.3/dist/purify.min.js';
import { marked } from 'https://cdn.jsdelivr.net/npm/marked/marked.min.js';
```

```
export async function renderBlogList({mount}){
  const el = document.createElement('div');
  el.innerHTML = '<h2>Blog</h2><div id="posts" class="post-list">Cargando...</div>';
  mount.appendChild(el);
  try{
    const idx = await listPostsIndex();
    const container = el.querySelector('#posts');
    container.innerHTML = '';
    for(const pfile of idx.posts){
      const p = await loadPostByFile(pfile.file);
      const item = document.createElement('div');
      item.className = 'post-item';
      item.innerHTML = `<h3><a data-link href="/blog/${p.slug}">${escapeHtml(p.title)}</a>
class="small">${escapeHtml(p.author)} • ${escapeHtml(p.date)} •
${escapeHtml(p.excerpt||'')}</div>`;
      container.appendChild(item);
    }
  }catch(err){
    mount.innerHTML = `<div class="error">No se pudo cargar la lista de posts:
${err.message}</div>`;
  }
}

export async function renderBlogDetail({mount, slug}){
  const el = document.createElement('article');
  el.innerHTML = '<div id="post-root">Cargando post...</div>';
  mount.appendChild(el);
  try{
    const p = await loadPostBySlug(slug);
    const html = (p.contentFormat === 'markdown') ? marked.parse(p.content || '') : p.cont
'';
    const clean = DOMPurify.sanitize(html);
    const inner = document.getElementById('post-root');
    inner.innerHTML = `<h1>${escapeHtml(p.title)}</h1><div class="small">${escapeHtml(p.au
${escapeHtml(p.date)}</div><div class="preview">${clean}</div>`;
  }catch(err){
    mount.innerHTML = `<div class="error">Error cargando post: ${err.message}</div>`;
  }
}

function escapeHtml(s){ return String(s||'').replace(/[&<>"']/g, function(m){return
{'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;","'":'&#39;'}[m];}); }""",
  "src/pages/contact.js": """export async function renderContact({mount, config}){
  const el = document.createElement('div');
  el.innerHTML = `
    <h2>Contacto</h2>
    <form id="contact-form" class="contact-form" novalidate>
      <div class="form-row"><label for="name">Nombre</label><input id="name" name="name"
type="text" required></div>
      <div class="form-row"><label for="email">Email</label><input id="email" name="email"
type="email" required></div>
      <div class="form-row"><label for="subject">Asunto</label><input id="subject" name="s
type="text"></div>
      <div class="form-row"><label for="message">Mensaje</label><textarea id="message"
name="message" required></textarea></div>
      <div class="controls">
        <button class="btn" type="submit">Enviar</button>
        <button class="btn ghost" type="button" id="clear">Limpiar</button>
      </div>
      <div id="contact-status" role="status" aria-live="polite"></div>
```

```
    </form>
    <p class="small">Este formulario usa su proveedor configurado en
<code>/data/config.json</code>.</p>
  `;
  mount.appendChild(el);
  const form = el.querySelector('#contact-form');
  const status = el.querySelector('#contact-status');
  const clear = el.querySelector('#clear');

  form.addEventListener('submit', async(e)=>{
    e.preventDefault();
    status.textContent = '';
    const data = {
      name: form.name.value,
      email: form.email.value,
      subject: form.subject.value,
      message: form.message.value
    };
    if(!data.email || !data.name || !data.message){
      status.textContent = 'Por favor completa los campos requeridos.';
      status.className = 'error';
      return;
    }
    try{
      // read config for endpoint
      const cfg = await fetch('/data/config.json').then(r=>r.json());
      if(cfg.email && cfg.email.provider === 'formspree' && cfg.email.formEndpoint){
        // Formspree expects form-encoded post to endpoint
        const resp = await fetch(cfg.email.formEndpoint, {
          method:'POST',
          headers:{ 'Content-Type':'application/json', 'Accept':'application/json' },
          body: JSON.stringify({ name: data.name, email: data.email, subject: data.subject
message: data.message })
        });
        if(resp.ok){
          status.textContent = 'Mensaje enviado. Gracias.';
          status.className = 'success';
          form.reset();
        }else{
          const body = await resp.json().catch(()=>({}));
          status.textContent = 'Fallo al enviar: ' + (body.error || resp.statusText ||
resp.status);
          status.className = 'error';
        }
      }else{
        // serverless or no provider configured
        status.innerHTML = 'No hay proveedor configurado. Vea README para opciones.';
        status.className = 'error';
      }
    }catch(err){
      status.textContent = 'Error en envío: ' + err.message;
      status.className = 'error';
    }
  });

  clear.addEventListener('click', ()=>form.reset());
}""",
  "src/admin/index.js": """import { listPostsIndex, loadPostByFile, downloadJSON } from
'../lib/storage.js';
import { loadSchema } from '../lib/schema.js';
```

```
export async function renderAdmin({mount, config}){
  const root = document.createElement('div');
  root.innerHTML = '<h2>Admin</h2><div id="admin-area"></div>';
  mount.appendChild(root);
  const area = root.querySelector('#admin-area');

  // Show config and routes editing
  const cfgSection = document.createElement('section');
  cfgSection.innerHTML = '<h3>Configuración global</h3><div class="controls"><button
id="edit-config" class="btn">Editar config.json</button><button id="download-config" class
ghost">Descargar config.json</button></div><div id="config-editor" class="preview small"><
  area.appendChild(cfgSection);

  document.getElementById('edit-config').addEventListener('click', async ()=>{
    const cfg = await fetch('/data/config.json').then(r=>r.text());
    showJsonEditor('config.json', cfg, 'config');
  });
  document.getElementById('download-config').addEventListener('click', async ()=>{
    const cfg = await fetch('/data/config.json').then(r=>r.json());
    downloadJSON('config.json', cfg);
  });

  // Posts list and editor
  const postsSection = document.createElement('section');
  postsSection.innerHTML = '<h3>Posts</h3><div class="controls"><button id="new-post"
class="btn">Nuevo Post</button><button id="reload-posts" class="btn
ghost">Recargar</button></div><div id="posts-admin" class="post-list"></div>';
  area.appendChild(postsSection);

  async function loadPosts(){
    const box = postsSection.querySelector('#posts-admin');
    box.innerHTML = 'Cargando...';
    try{
      const idx = await listPostsIndex();
      box.innerHTML = '';
      for(const p of idx.posts){
        const post = await loadPostByFile(p.file);
        const item = document.createElement('div');
        item.className = 'post-item';
        item.innerHTML = `<strong>${escapeHtml(post.title)}</strong> <div
class="small">${escapeHtml(post.slug)} • ${escapeHtml(post.date || '')}</div><div
class="controls"><button class="btn view" data-file="${p.file}">Ver</button><button class=
ghost edit" data-file="${p.file}">Editar</button><button class="btn ghost download"
data-file="${p.file}">Descargar</button></div>`;
        box.appendChild(item);
      }
      box.querySelectorAll('.view').forEach(b=>b.addEventListener('click', async ev=>{
        const f = ev.target.dataset.file;
        const p = await loadPostByFile(f);
        alert('Vista rápida:\\n' + JSON.stringify(p, null, 2));
      }));
      box.querySelectorAll('.edit').forEach(b=>b.addEventListener('click', async ev=>{
        const f = ev.target.dataset.file;
        const txt = await fetch('/data/posts/' + f).then(r=>r.text());
        showJsonEditor(f, txt, 'post');
      }));
      box.querySelectorAll('.download').forEach(b=>b.addEventListener('click', async ev=>{
        const f = ev.target.dataset.file;
        const json = await fetch('/data/posts/' + f).then(r=>r.json());
        downloadJSON(f, json);
      }));
```

```
    }catch(err){
      box.innerHTML = '<div class="error">Error: ' + err.message + '</div>';
    }
  }
  postsSection.querySelector('#reload-posts').addEventListener('click', loadPosts);
  postsSection.querySelector('#new-post').addEventListener('click', async ()=>{
    // build a new post skeleton via schema defaults if present
    const schema = await loadSchema('post.schema').catch(()=>null);
    const obj = {
      slug: 'nuevo-post-' + Date.now(),
      title: 'Título del post',
      author: config && config.admin && config.admin.user ? config.admin.user : 'Admin',
      date: new Date().toISOString().slice(0,10),
      tags: [],
      excerpt: '',
      contentFormat: 'markdown',
      content: '# Nuevo post\\nEscribe aquí...'
    };
    showJsonEditor('post-new.json', JSON.stringify(obj, null, 2), 'post');
  });

  // simple JSON editor modal/section
  async function showJsonEditor(filename, text, type){
    const modal = document.createElement('div');
    modal.className = 'preview';
    modal.innerHTML = `
      <h4>Editor: ${escapeHtml(filename)}</h4>
      <textarea id="json-editor"
style="width:100%;min-height:260px;font-family:monospace">${escapeHtml(text)}</textarea>
      <div class="controls">
        <button id="validate-save" class="btn">Validar & Descargar</button>
        <button id="import-json" class="btn ghost">Importar JSON (archivo)</button>
        <button id="close-editor" class="btn ghost">Cerrar</button>
      </div>
      <div id="editor-status" class="small"></div>
    `;
    area.prepend(modal);
    const editor = modal.querySelector('#json-editor');
    modal.querySelector('#close-editor').addEventListener('click', ()=>modal.remove());
    modal.querySelector('#import-json').addEventListener('click', ()=>fileImport(editor));
    modal.querySelector('#validate-save').addEventListener('click', async ()=>{
      const valTxt = editor.value;
      try{
        const obj = JSON.parse(valTxt);
        // basic schema validation with Ajv (loaded in index.html)
        let schemaName = 'config';
        if(type === 'post') schemaName = 'post';
        const schema = await fetch('/schemas/' + schemaName +
'.schema.json').then(r=>r.json()).catch(()=>null);
        if(schema && window.Ajv){
          const ajv = new window.Ajv();
          const valid = ajv.validate(schema, obj);
          if(!valid){
            modal.querySelector('#editor-status').innerHTML = '<div class="error">Errores:
JSON.stringify(ajv.errors, null, 2) + '</div>';
            return;
          }
        }
        // simulated save: download JSON and show success (for GH Pages flows)
        const outName = filename.startsWith('post-') || filename.endsWith('.json') ? filen
filename + '.json';
```

```
        downloadJSON(outName, obj);
        modal.querySelector('#editor-status').innerHTML = '<div class="success">Descargado
archivo en /data/posts/ y actualice /data/posts/index.json para persistir.</div>';
      }catch(err){
        modal.querySelector('#editor-status').innerHTML = '<div class="error">JSON inválid
err.message + '</div>';
      }
    });
  }

  async function fileImport(editorEl){
    const input = document.createElement('input');
    input.type = 'file';
    input.accept = '.json,application/json';
    input.onchange = async (e)=>{
      const f = e.target.files[0];
      if(!f) return;
      const txt = await f.text();
      editorEl.value = txt;
    };
    input.click();
  }

  loadPosts();
} // end renderAdmin

function escapeHtml(s){ return String(s||'').replace(/[&<>"']/g, function(m){return
{'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[m];}); }""",
  "src/lib/README.md": """Helpers usados por la UI:
- storage.js: lectura de JSON desde /data y utilidades para 'guardar' simulando descarga.
- schema.js: carga esquemas desde /schemas para validar con Ajv en cliente.
""",
  "data/config.json": """{
  "siteName": "Mi Sitio Estático",
  "brand": {
    "logo": "/public/assets/logo.svg",
    "primaryColor": "#1f6feb"
  },
  "seo": {
    "title": "Inicio - Mi Sitio Estático",
    "description": "Sitio estático tipo WP con JSON"
  },
  "email": {
    "provider": "formspree",
    "formEndpoint": "https://formspree.io/f/your-form-id"
  },
  "admin": {
    "enabled": true,
    "password": "cambia-esto",
    "user": "admin"
  },
  "footer": "Mi Sitio Estático • Demo",
  "home": {
    "blocks": [
      {
        "type": "hero",
        "title": "Bienvenido a la demo",
        "subtitle": "Sitio estático gestionado con JSON",
        "cta": { "text": "Ver blog", "link": "/blog" }
      },
      {
```

```
        "type": "cards",
        "items": [
          { "title": "Leer posts", "subtitle": "Ver entradas del blog", "link": "/blog" },
          { "title": "Contacto", "subtitle": "Escríbenos", "link": "/contacto" }
        ]
      },
      {
        "type": "posts",
        "title": "Últimos posts",
        "limit": 3
      }
    ]
  }
}
""",
  "data/routes.json": """{
  "routes": [
    { "path": "/", "page": "home" },
    { "path": "/blog", "page": "blog-list" },
    { "path": "/blog/:slug", "page": "blog-detail" },
    { "path": "/contacto", "page": "contact" },
    { "path": "/admin", "page": "admin" }
  ]
}
""",
  "data/posts/index.json": """{
  "posts": [
    { "file": "post-001.json", "slug": "primer-post" },
    { "file": "post-002.json", "slug": "segundo-post" }
  ]
}
""",
  "data/posts/post-001.json": """{
  "slug": "primer-post",
  "title": "Primer Post",
  "author": "Bernardo",
  "date": "2025-10-13",
  "tags": ["demo", "json"],
  "excerpt": "Ejemplo de post guardado en JSON.",
  "contentFormat": "markdown",
  "content": "# Hola\\nEste es el **primer post** en formato Markdown.\\n\\n- Punto 1\\n-
}
""",
  "data/posts/post-002.json": """{
  "slug": "segundo-post",
  "title": "Segundo Post de Ejemplo",
  "author": "María",
  "date": "2025-10-14",
  "tags": ["ejemplo"],
  "excerpt": "Otro post para demostrar la lista.",
  "contentFormat": "markdown",
  "content": "Contenido del segundo post. Puedes editarlo desde el admin y descargar el JS
}
""",
  "schemas/config.schema.json": """{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Config",
  "type": "object",
  "required": ["siteName", "brand", "seo", "admin"],
  "properties": {
    "siteName": {"type":"string"},
```

```
    "brand": {
      "type":"object",
      "properties": {
        "logo": {"type":"string"},
        "primaryColor": {"type":"string"}
      }
    },
    "seo": {
      "type":"object",
      "properties": {
        "title": {"type":"string"},
        "description": {"type":"string"}
      }
    },
    "email": {
      "type":"object",
      "properties": {
        "provider": {"type":"string"},
        "formEndpoint": {"type":"string"}
      }
    },
    "admin": {
      "type":"object",
      "properties": {
        "enabled": {"type":"boolean"},
        "password": {"type":"string"},
        "user": {"type":"string"}
      },
      "required": ["enabled","password"]
    },
    "home": {"type":"object"}
  }
}
""",
  "schemas/routes.schema.json": """{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Routes",
  "type": "object",
  "properties": {
    "routes": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "path": {"type":"string"},
          "page": {"type":"string"}
        },
        "required":["path","page"]
      }
    }
  }
}
""",
  "schemas/post.schema.json": """{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Post",
  "type": "object",
  "required": ["slug","title","author","date","contentFormat","content"],
  "properties": {
    "slug": {"type":"string","pattern":"^[a-z0-9\\-]+$"},
    "title": {"type":"string"},
```

```
    "author": {"type":"string"},
    "date": {"type":"string","format":"date"},
    "tags": {"type":"array","items":{"type":"string"}},
    "excerpt": {"type":"string"},
    "contentFormat": {"type":"string","enum":["markdown","html"]},
    "content": {"type":"string"}
  }
}
""",
  "README.md": """# Mi Sitio Estático (WP-like con JSON)

Este repo contiene un sitio web estático gestionable sin base de datos. Toda la persistenc
realiza en archivos JSON bajo `/data/`. El cliente (JS) lee estos JSON y renderiza páginas
Blog, Post detail y Contacto. Hay un modo Admin ligero para editar JSON en el navegador y
cambios (workflow para persistencia).

Estructura principal:
- /public: archivos estáticos (index.html, assets, sitemap, robots)
- /src: código fuente JS (componentes, páginas, admin, helpers)
- /data: contenido en JSON (config.json, routes.json, posts/*.json)
- /schemas: JSON Schemas para validación
- /scripts: utilidades Node para build/local-save
- README con instrucciones

Arrancar local (rápido)
1. Instala dependencias:
   npm install

2. Ejecuta servidor estático (sirve el repo para que /data sea accesible):
   npm run dev

3. Abre en el navegador:
   http://localhost:5000/public/index.html
   - Navega a /, /blog, /contacto.
   - Admin: agrega `?admin=1&password=cambia-esto` en la URL para activar admin (cambia la
contraseña en /data/config.json).

Notas:
- Para persistencia real (escritura de archivos JSON) hay dos caminos:
  1) Local (desarrollo): usar el script Node `scripts/save-post-local.js` para escribir
directamente a /data/posts/ (ver sección).
  2) GitHub Pages / Netlify sin backend: el admin descarga JSON editados; sube manualmente
vía PR (guiado en README).

Cómo crear/editar posts (UI admin)
- En el sitio, añade `?admin=1&password=TU_PASS` y abre Admin.
- Usa "Nuevo Post" para crear una plantilla. Edita JSON en el editor.
- "Validar & Descargar" validará con el JSON Schema y descargará el archivo .json.
- Para publicar en GH Pages: sube el archivo descargado a `/data/posts/` y actualiza
`/data/posts/index.json`. Para flujos manuales, después crea un PR.

Validación
- Los esquemas están en `/schemas/*.schema.json`.
- En admin se utiliza Ajv (cargado desde CDN) para validar antes de permitir descarga.

Contacto
- Usa Formspree por defecto. Configura `data/config.json.email.formEndpoint` con tu endpoi
Formspree.
- El form envía JSON y muestra estado (éxito/error).
- Alternativa serverless: hay ejemplo de endpoint en `.github/workflows` y `scripts/` para
Worker; ver sección Deploy.
```

```
Guardar desde UI (persistencia)
- Para repositorios sin backend (GitHub Pages) la función "Guardar" descargará el JSON. Su
manualmente al repo por PR.
- Para usos locales, puede ejecutarse:
   node scripts/save-post-local.js --file data/posts/post-new.json
  Esto crea el archivo localmente. (Útil en entorno dev)

Scripts y build
- npm run dev -> inicia servidor estático (puede usarse con curl/npx)
- npm run build -> copia /public a /dist (script Node)

Seguridad
- No incluyas credenciales en el repo.
- Para producción con serverless, usa variables de entorno (.env) y no las comites.

Deploy
- GitHub Pages: publicar contenido de `/public` en gh-pages o usar GitHub Actions (workflo
incluido).
- Netlify/Vercel: desplegar la carpeta `/public`. Si necesitas serverless, añade función p
/api/send-email y configura variables de entorno.

GitHub Actions / CI
- Hay un workflow de ejemplo en `.github/workflows/ci.yml` que ejecuta `npm run build` y p
publicar en gh-pages (configurar secretos).

Archivos clave
- /data/config.json -> branding, admin.password, email provider, home blocks.
- /data/routes.json -> rutas y páginas.
- /data/posts/*.json -> posts individuales.
- /data/posts/index.json -> índice de posts (cliente lo usa para listar).

Limitaciones
- El editor no escribe directamente al disco por motivos de seguridad en navegadores.
- Para escritura remota automatizada se necesitan endpoints seguros (serverless/CI).

Despliegue rápido en GitHub Pages
1. Crear un branch gh-pages o configurar Actions para desplegar /public a gh-pages.
2. Asegúrate de que /public/index.html y /data estén incluidos en la rama desplegada.

Gracias — usa este proyecto como base para expandir funcionalidad (i18n, autenticación rea
serverless).
""",
  ".gitignore": """node_modules
dist
.env
.DS_Store
coverage
*.log
""",
  ".env.example": """# Variables sensibles (no incluir en repo)
# FORM_SUPER_SECRET=...
# For serverless email provider (if using Cloudflare Worker / AWS Lambda)
# EMAIL_API_KEY=
""",
  "package.json": """{
  "name": "mi-sitio-estatico",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "npx http-server -p 5000",
```

```
    "build": "node scripts/build.js"
  },
  "dependencies": {},
  "devDependencies": {
    "http-server": "^14.1.1"
  },
  "engines": {
    "node": ">=14"
  }
}
""",
  "scripts/build.js": """// simple build: copy /public to /dist (node)
const fs = require('fs');
const path = require('path');

function copyDir(src, dest){
  if(!fs.existsSync(dest)) fs.mkdirSync(dest, {recursive:true});
  const entries = fs.readdirSync(src, {withFileTypes:true});
  for(const e of entries){
    const sp = path.join(src, e.name);
    const dp = path.join(dest, e.name);
    if(e.isDirectory()){
      copyDir(sp, dp);
    }else{
      fs.copyFileSync(sp, dp);
    }
  }
}

const from = path.join(__dirname, '..', 'public');
const to = path.join(__dirname, '..', 'dist');

try{
  if(fs.existsSync(to)) {
    fs.rmSync(to, {recursive:true, force:true});
  }
  copyDir(from, to);
  // also copy data and schemas for static site
  copyDir(path.join(__dirname, '..', 'data'), path.join(to, 'data'));
  copyDir(path.join(__dirname, '..', 'schemas'), path.join(to, 'schemas'));
  console.log('Build complete -> dist/');
}catch(err){
  console.error('Build failed', err);
  process.exit(1);
}
""",
  "scripts/save-post-local.js": """// Usage: node scripts/save-post-local.js
--file=path/to/file.json
// Writes a JSON file to disk (useful for local dev to persist admin downloads)
const fs = require('fs');
const path = require('path');

const args = process.argv.slice(2);
const opts = {};
args.forEach(a=>{
  const m = a.match(/^--([^=]+)=(.+)$/);
  if(m) opts[m[1]] = m[2];
});
if(!opts.file){
  console.error('Usage: node scripts/save-post-local.js --file=path/to/file.json');
  process.exit(1);
```

```
}
const inputPath = path.resolve(opts.file);
if(!fs.existsSync(inputPath)){
  console.error('File not found:', inputPath);
  process.exit(1);
}
const dest = path.join(process.cwd(), 'data', 'posts', path.basename(inputPath));
fs.copyFileSync(inputPath, dest);
console.log('Saved to', dest);
// optionally update index.json
const idxPath = path.join(process.cwd(),'data','posts','index.json');
let idx = {posts:[]};
if(fs.existsSync(idxPath)){
  idx = JSON.parse(fs.readFileSync(idxPath,'utf8'));
}
const json = JSON.parse(fs.readFileSync(inputPath,'utf8'));
const entry = { file: path.basename(inputPath), slug: json.slug ||
path.basename(inputPath).replace(/\\.json$/,'') };
idx.posts.unshift(entry);
fs.writeFileSync(idxPath, JSON.stringify(idx, null, 2));
console.log('Updated index.json');""",
  ".github/workflows/ci.yml": """name: CI / Build

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: '18'
      - name: Install
        run: npm ci
      - name: Build
        run: npm run build
      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: site-dist
          path: dist
""",
  "public/README_PLACEHOLDER.txt": """Este directorio se sirve como público. Abra
/public/index.html en su navegador (o use npm run dev).""",
  "src/README.md": """Código fuente modular:
- main.js: inicializa y enruta.
- /lib: helpers para cargar JSON y validar.
- /pages: renderizadores para home, blog, contacto.
- /admin: UI admin que valida con schemas y permite descargar JSON.
""",
  "LICENSE": """MIT License

Copyright (c) 2025

Permission is hereby granted, free of charge, to any person obtaining a copy
```

```
of this software ... (recorta en producción)
"""
}
```